

Assignment -4

Aim

To design and implement a C++ program that converts a given Regular Expression into a Non-Deterministic Finite Automaton (NFA) using Thompson's Construction method.

Theory

A Regular Expression (RE) is a sequence of characters that defines a search pattern and is used to describe regular languages. Regular expressions are widely used in compiler design for specifying the lexical structure of programming languages.

A Non-Deterministic Finite Automaton (NFA) is a finite automaton where for a particular input symbol, the machine may move to any number of possible next states. An NFA may also contain ϵ (epsilon) transitions, which allow the automaton to change states without consuming any input symbol.

The conversion of a Regular Expression to NFA is usually done using Thompson's Construction Algorithm. In this method, small NFAs are constructed for basic symbols and then combined to form larger NFAs using operators like:

- Concatenation (.)
- Union (|)
- Kleene Star (*)

The generated NFA consists of a start state, final state, and transitions that represent the structure of the regular expression. This process is an important step in building lexical analyzers in compilers.

Algorithm

1. Start the program.
2. Input the regular expression.
3. Convert the regular expression into postfix form if necessary.
4. Use Thompson's construction rules to build NFA:
 - For each symbol, create a basic NFA with two states.
 - For concatenation, connect the final state of the first NFA to the start state of the second NFA.
 - For union ($|$), create a new start and final state and connect both NFAs using ϵ transitions.
 - For Kleene star ($*$), create loops using ϵ transitions.
5. Store states and transitions.
6. Display the generated NFA transitions.
7. Stop the program.

Program (C++)

```
#include <iostream>
```

```
#include <stack>
```

```
#include <string>
```

```
using namespace std;
```

```
int state = 0;
```

```
struct NFA {
```

```
    int start;
```

```
    int end;
```

```
};
```

```
stack<NFA> st;
```

```
void symbolNFA(char c) {
```

```
    NFA n;
```

```
    n.start = state++;
```

```
    n.end = state++;
```

```
    cout << n.start << " -- " << c << " --> " << n.end << endl;
```

```
    st.push(n);
```

```
}
```

```
void concat() {
```

```
    NFA n2 = st.top(); st.pop();
```

```
    NFA n1 = st.top(); st.pop();
```

```
cout << n1.end << " -- e --> " << n2.start << endl;
```

```
NFA n;
```

```
n.start = n1.start;
```

```
n.end = n2.end;
```

```
st.push(n);
```

```
}
```

```
void unionNFA() {
```

```
    NFA n2 = st.top(); st.pop();
```

```
    NFA n1 = st.top(); st.pop();
```

```
    NFA n;
```

```
    n.start = state++;
```

```
    n.end = state++;
```

```
    cout << n.start << " -- e --> " << n1.start << endl;
```

```
    cout << n.start << " -- e --> " << n2.start << endl;
```

```
    cout << n1.end << " -- e --> " << n.end << endl;
```

```
    cout << n2.end << " -- e --> " << n.end << endl;
```

```
    st.push(n);  
}
```

```
void kleene() {  
    NFA n1 = st.top(); st.pop();
```

```
    NFA n;  
    n.start = state++;  
    n.end = state++;
```

```
    cout << n.start << " -- e --> " << n1.start << endl;  
    cout << n.start << " -- e --> " << n.end << endl;  
    cout << n1.end << " -- e --> " << n1.start << endl;  
    cout << n1.end << " -- e --> " << n.end << endl;
```

```
    st.push(n);  
}
```

```
int main() {
```

```
    string re;
```

```
cout << "Enter postfix regular expression: ";
```

```
cin >> re;
```

```
cout << "\nNFA Transitions:\n";
```

```
for(char c : re) {
```

```
    if(c == '.')
```

```
        concat();
```

```
    else if(c == '|')
```

```
        unionNFA();
```

```
    else if(c == '*')
```

```
        kleene();
```

```
    else
```

```
        symbolNFA(c);
```

```
}
```

```
NFA result = st.top();
```

```
cout << "\nStart State: " << result.start << endl;

cout << "Final State: " << result.end << endl;

return 0;

}
```

Output

Clear

Enter postfix regular expression: ab.

NFA Transitions:
0 -- a --> 1
2 -- b --> 3
1 -- e --> 2

Start State: 0
Final State: 3

=== Code Execution Successful ===

Conclusion

(Regular Expression $\rightarrow$ NFA)

The program for converting a Regular Expression into a Non-Deterministic Finite Automaton (NFA) was successfully designed and implemented using Thompson's construction method. The program takes a regular expression as input and generates the corresponding NFA with appropriate states and transitions. This experiment helped in understanding how regular expressions are converted into automata during the lexical analysis phase of a compiler.

Result

Thus, the C++ program to convert a given Regular Expression into a Non-Deterministic Finite Automaton (NFA) was implemented and executed successfully.